Questions of the day

- How can we use DFS to resolve a set of prerequisites?
- How can we determine the *bandwidth* of an arbitrarily complex network?

More graph algorithms: topological sorting and network flows

William Hendrix

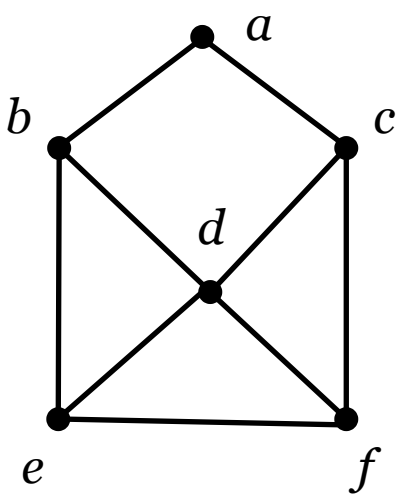
Lecture 11

Outline

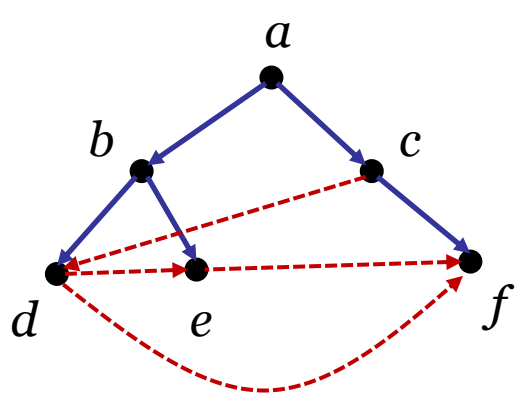
- Review
  - Traversals
  - Dijkstra's Algorithm
  - Prim's and Kruskal's Algorithms
- Topological sorting
- Max-flow and min-cut problems
  - Ford-Fulkerson

Review: traversal trees

- Traversal tree
  - Set of edges traversed by BFS or DFS,
  - Augmented with other edges of graph
- BFS and DFS trees have different properties
- BFS: all non-tree edges are "cross-edges"
  - Same level or one level down
- DFS: all non-tree edges are "back-edges"
  - Point to ancestor
- **Example**

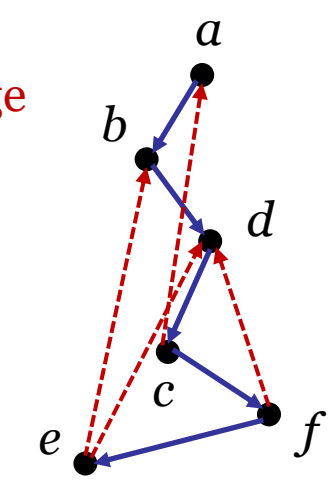


BFS



Tree edge
Non-tree edge

DFS



Review: weighted graph algorithms

- **Dijkstra's Algorithm:** weighted graph distance
  - Label initial vertex with distance 0
  - While not at goal (or vertices remain):
    - Find vertex with min label
    - Label neighbors with parent dist + edge weight
      - Unless existing label is better
  - Can be modified to return path
  - *Complexity:* $\Theta(n + m \lg n)$ or $\Theta(m \lg n)$
- **Prim's Algorithm:** minimum spanning tree (MST)
  - Label initial vertex with 0
  - While vertices remain:
    - Find vertex with min label
    - Add edge of this vertex to its parent to MST
    - Label neighbors with edge weight
      - Unless existing label is better

Review: weighted graph algorithms

- **Kruskal's Algorithm:** minimum spanning tree
  - Sort edges by weight
  - Uses union find!
  - Iterate through edges:
    - If endpoints not in same group, union and add to MST

Prim's analysis

```
Input: G = (V, E): graph in which to find MST
Input: n, m: order and size of G
Output: Minimum spanning tree of G
edge = Array(n)
Let heap be a heap where every vertex has value ∞
Choose v_0 and decrease its value in heap to 0
for i = 1 to n do
    v = heap.DeleteMin()
    If v ≠ v_0, add (v, edge[v]) to MST
    for u ∈ N(v) do
        if u ∈ heap and d(u, v) < value(u) then
            Decrease value of u to d(u, v) in heap
            edge[u] = v
        end
    end
end
return MST
```

Prim's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum spanning tree of G
$\Theta(1)$	$edge = \text{Array}(n)$
$\Theta(n)$	Let $heap$ be a heap where every vertex has value ∞
$\Theta(\lg n)$	Choose v_0 and decrease its value in $heap$ to 0
	for $i = 1$ to n **do**
	$\quad v = heap.\text{DeleteMin}()$
	$\quad$ If $v \neq v_0$, add $(v, edge[v])$ to MST
	$\quad$ **for** $u \in N(v)$ **do**
$\Theta(1)$	$\quad\quad$ **if** $u \in heap$ and $d(u, v) < \text{value}(u)$ **then**
$\Theta(\lg n)$	$\quad\quad\quad$ Decrease value of u to $d(u, v)$ in $heap$
$\Theta(1)$	$\quad\quad\quad$ $\text{edge}[u] = v$
	$\quad\quad$ **end**
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Prim's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum spanning tree of G
$\Theta(1)$	$edge = \text{Array}(n)$
$\Theta(n)$	Let $heap$ be a heap where every vertex has value ∞
$\Theta(\lg n)$	Choose v_0 and decrease its value in $heap$ to 0
	for $i = 1$ to n **do**
	$\quad v = heap.\text{DeleteMin}()$
	$\quad$ If $v \neq v_0$, add $(v, edge[v])$ to MST
$\Theta(\deg(v))$ iters	$\quad$ **for** $u \in N(v)$ **do**
	$\quad\quad$ **if** $u \in heap$ and $d(u, v) < \text{value}(u)$ **then**
$\Theta(\lg n)$	$\quad\quad\quad$ Decrease value of u to $d(u, v)$ in $heap$
	$\quad\quad\quad$ $\text{edge}[u] = v$
	$\quad\quad$ **end**
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Prim's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum spanning tree of G
$\Theta(1)$	$edge = \text{Array}(n)$
$\Theta(n)$	Let $heap$ be a heap where every vertex has value ∞
$\Theta(\lg n)$	Choose v_0 and decrease its value in $heap$ to 0
$\Theta(n)$ iters	**for** $i = 1$ to n **do**
$\Theta(\lg n)$	$\quad v = heap.\text{DeleteMin}()$
$\Theta(1)$	$\quad$ If $v \neq v_0$, add $(v, edge[v])$ to MST
	$\quad$ **for** $u \in N(v)$ **do**
	$\quad\quad$ **if** $u \in heap$ and $d(u, v) < \text{value}(u)$ **then**
$\Theta(\deg(v) \lg n)$	$\quad\quad\quad$ Decrease value of u to $d(u, v)$ in $heap$
	$\quad\quad\quad$ $\text{edge}[u] = v$
	$\quad\quad$ **end**
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Prim's analysis

Complexity

Input: $G = (V, E)$: graph in which to find MST
Input: n, m: order and size of G
Output: Minimum spanning tree of G

$\Theta(1)$ — $edge = \text{Array}(n)$

$\Theta(n)$ — Let $heap$ be a heap where every vertex has value ∞

$\Theta(\lg n)$ — Choose v_0 and decrease its value in $heap$ to 0

$\Theta(m \lg n)$ —

```
for i = 1 to n do
    v = heap.DeleteMin()
    If v ≠ v_0, add (v, edge[v]) to MST
    for u ∈ N(v) do
        if u ∈ heap and d(u, v) < value(u) then
            Decrease value of u to d(u, v) in heap
            edge[u] = v
        end
    end
end
```

$\Theta(1)$ — **return** MST

Total: $\Theta(n + m \lg n)$

Kruskal's analysis

```
Input: G = (V, E): graph in which to find MST
Input: n, m: order and size of G
Output: Minimum Spanning Tree of G
E' = sort(E)
uf = UnionFind(n)
for i = 1 to m do
    (u, v) = E'[i]
    if uf.Find(u) ≠ uf.Find(v) then
        Add (u, v) to MST
        uf.Union(u, v)
    end
end
return MST
```

Kruskal's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum Spanning Tree of G
$\Theta(m \lg m)$	$E' = \text{sort}(E)$
$\Theta(n)$	$uf = \text{UnionFind}(n)$
	for $i = 1$ to m **do**
$\Theta(1)$	$\quad (u, v) = E'[i]$
$\Theta(\alpha(n))$	$\quad$ **if** $uf.\text{Find}(u) \neq uf.\text{Find}(v)$ **then**
$\Theta(1)$	$\quad\quad$ Add (u, v) to MST
$\Theta(\alpha(n))$	$\quad\quad uf.\text{Union}(u, v)$
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Kruskal's analysis

Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum Spanning Tree of G
$\Theta(m \lg m)$	$E' = \text{sort}(E)$
$\Theta(n)$	$uf = \text{UnionFind}(n)$
$\Theta(m)$	**for** $i = 1$ to m **do**
	$\quad (u, v) = E'[i]$
	$\quad$ **if** $uf.\text{Find}(u) \neq uf.\text{Find}(v)$ **then**
$\Theta(\alpha(n))$	$\quad\quad$ Add (u, v) to MST
	$\quad\quad uf.\text{Union}(u, v)$
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Kruskal's analysis

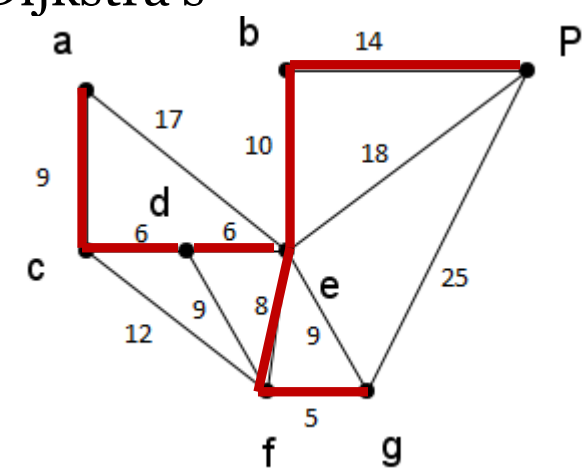
Complexity	
	Input: $G = (V, E)$: graph in which to find MST
	Input: n, m: order and size of G
	Output: Minimum Spanning Tree of G
$\Theta(m \lg m)$	$E' = \text{sort}(E)$
$\Theta(n)$	$uf = \text{UnionFind}(n)$
	for $i = 1$ to m **do**
	$\quad (u, v) = E'[i]$
	$\quad$ **if** $uf.\text{Find}(u) \neq uf.\text{Find}(v)$ **then**
$\Theta(m\alpha(n))$	$\quad\quad$ Add (u, v) to MST
	$\quad\quad uf.\text{Union}(u, v)$
	$\quad$ **end**
	end
$\Theta(1)$	**return** MST

Total complexity: $\Theta(n + m \lg m)$

MST analysis

- Prim's vs. Kruskal's
  - Prim's: $\Theta(n + m \lg n)$
  - Kruskal's: $\Theta(n + m \lg m)$
  - Asymptotically equivalent
    - Prim's typically faster for dense graphs
    - Similar time for sparse graphs
  - Linear space for both

- Note: Prim's algorithm very similar to Dijkstra's
- BUT: MST ≠ min distance

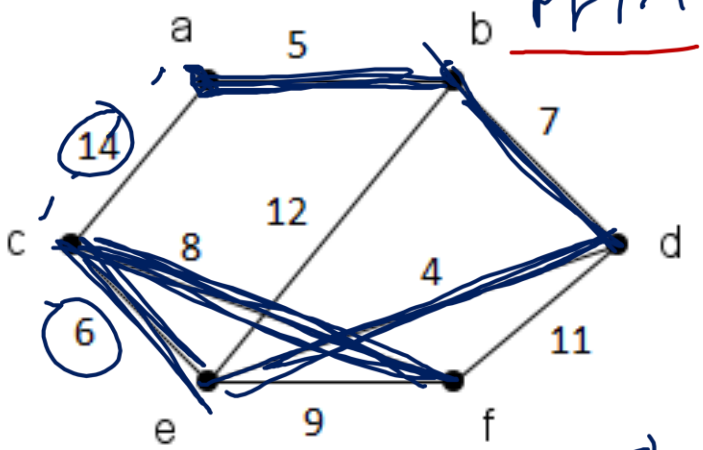


- MST Applications
  - Connecting spatial points
  - Travelling Salesman Problem
  - Clustering

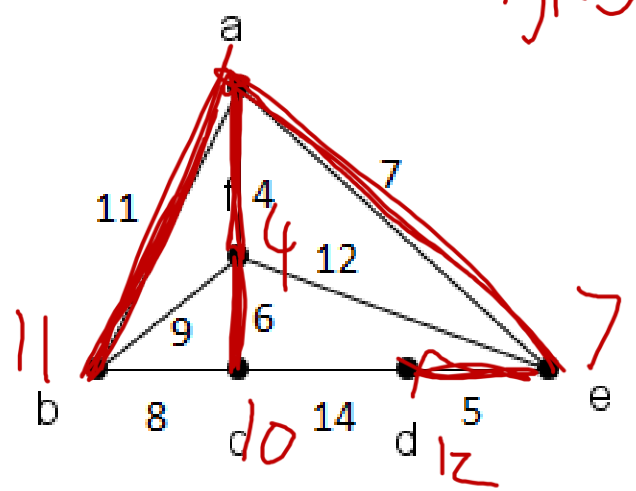
Graph algorithm exercise

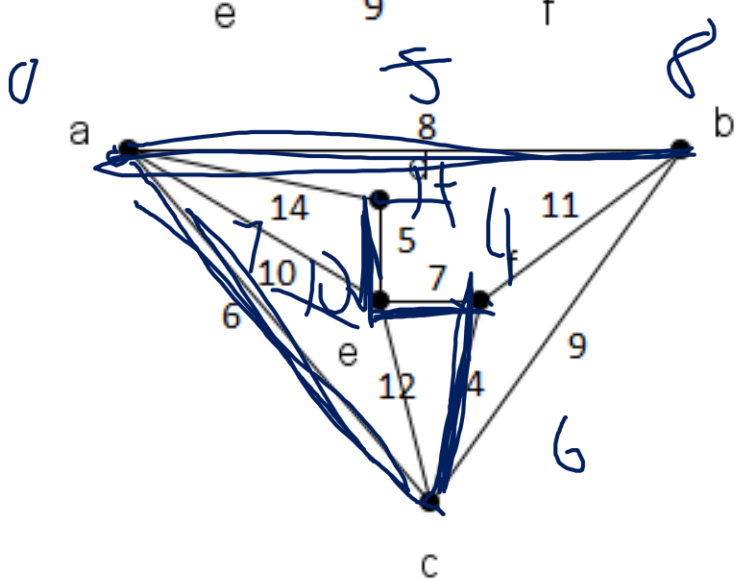
- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*

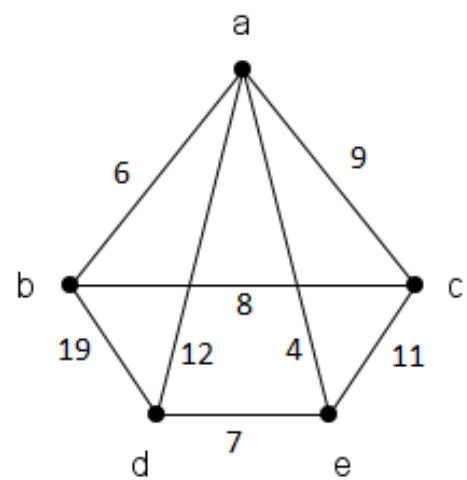
Prim's



Dijkstra's

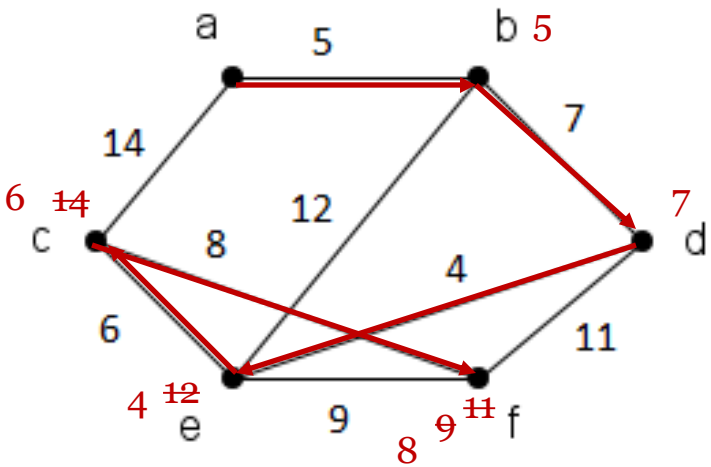


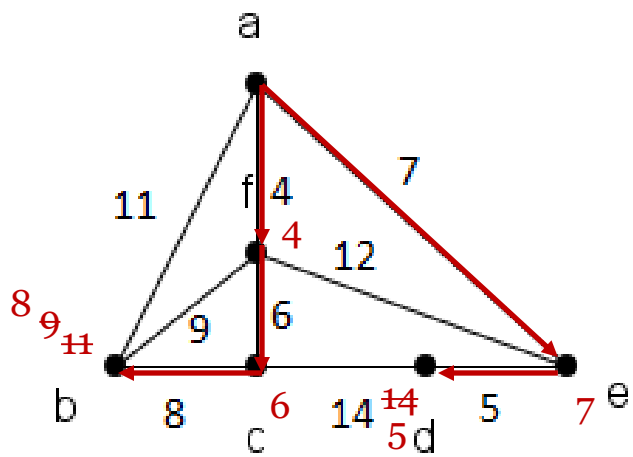


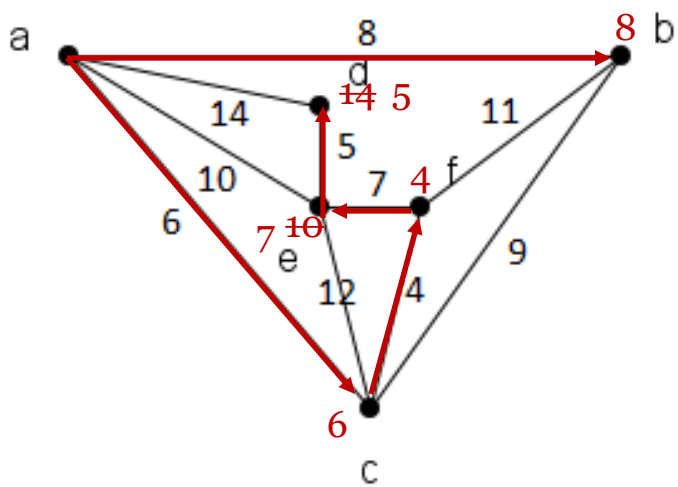


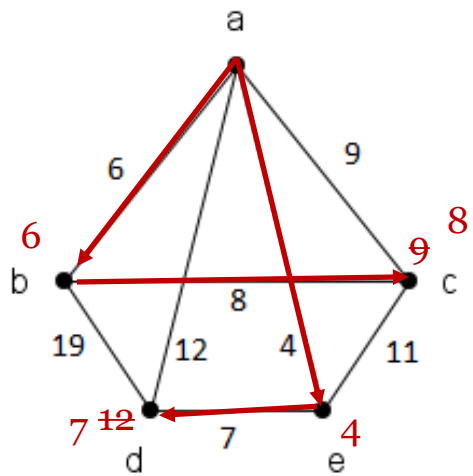
Prim's algorithm

- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*



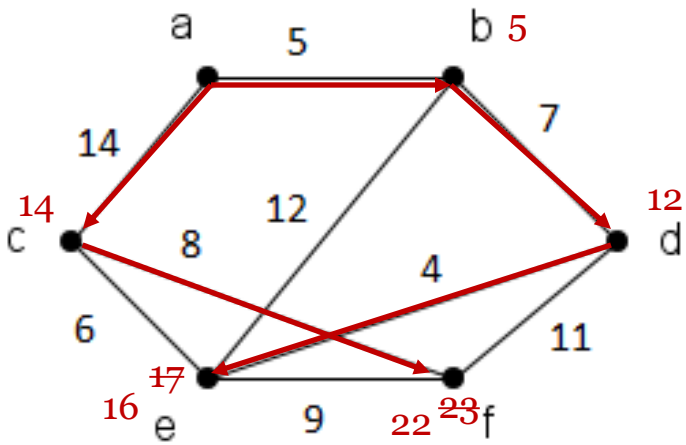


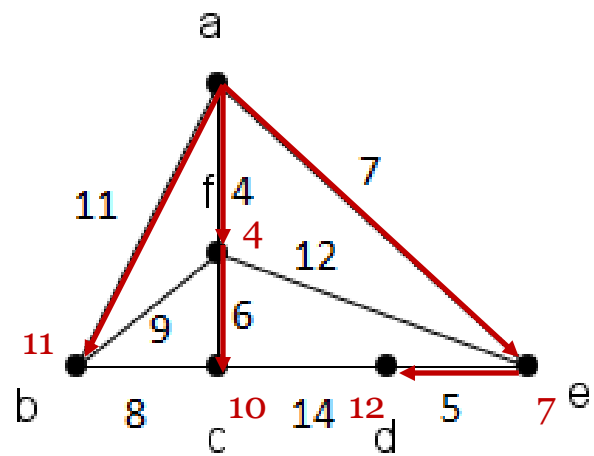


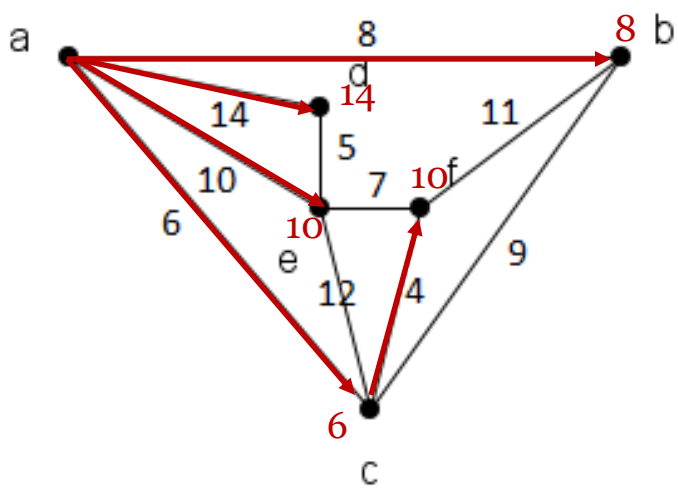


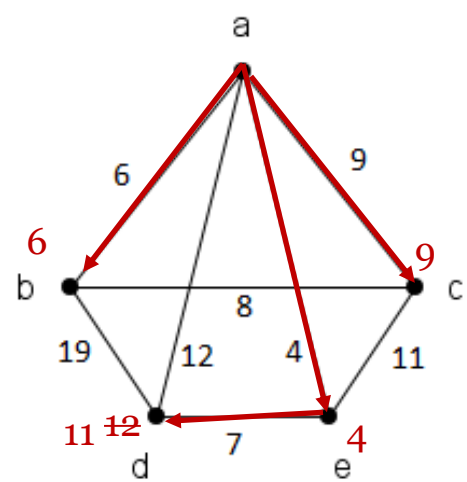
Dijkstra's algorithm

- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*



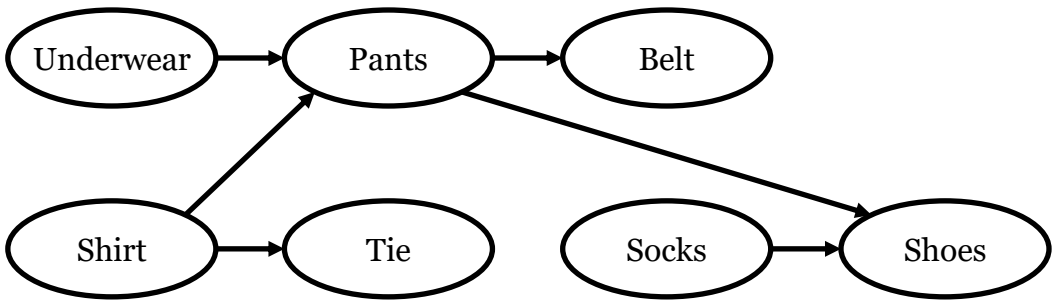






Directed acyclic graphs

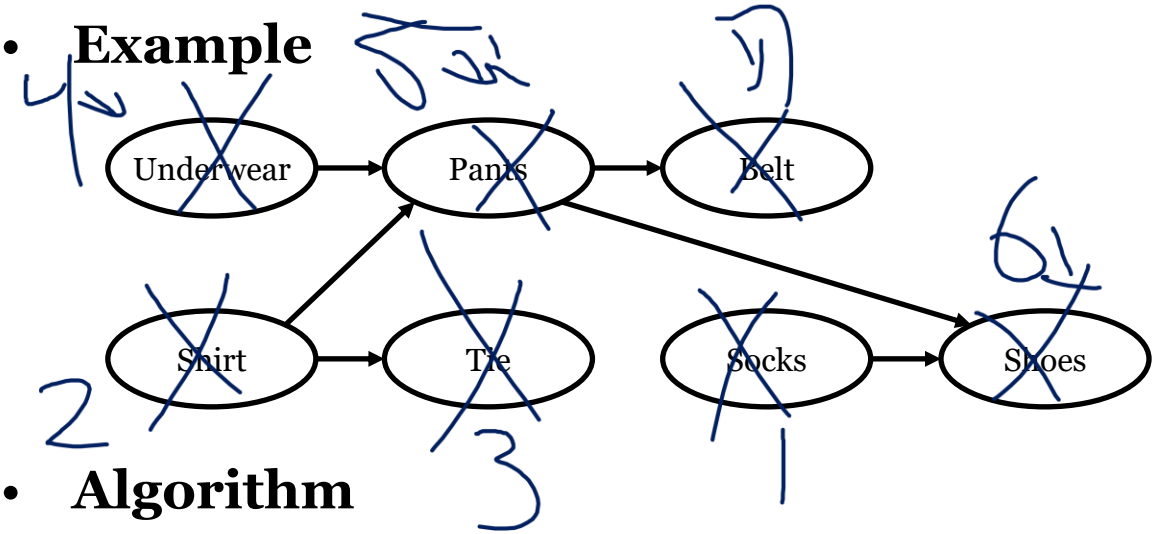
- Directed graphs with no cycles
- Sometimes used to represent processes or requirements
- **Example**



- *Important question*
  - How can I finish everything while meeting all requirements?

Topological sorting

- **Input:** directed acyclic graph (DAG)
- **Output:** vertices ordered so that all edges point right
- **Example**

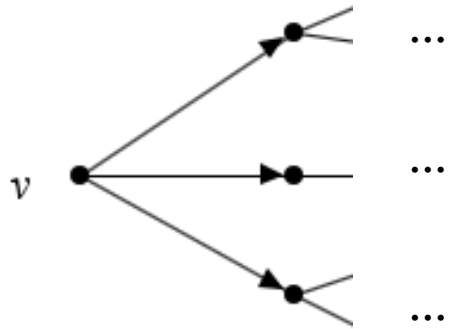


Belt
Shoes
Pants
U/W
Tie
Shirt
Socks

- **Algorithm**
  - Perform DFS
  - Add all vertices to stack when you mark them as explored
  - Return vertices in reverse order
- **Example (above)**
  - DFS: belt, shoes, pants, underwear, tie, shirt, socks
  - Reversed: socks, shirt, tie, underwear, pants, shoes, belt

Topological sorting correctness

- *Proof sketch*
  - Everything that depends on v is explored before v is finished

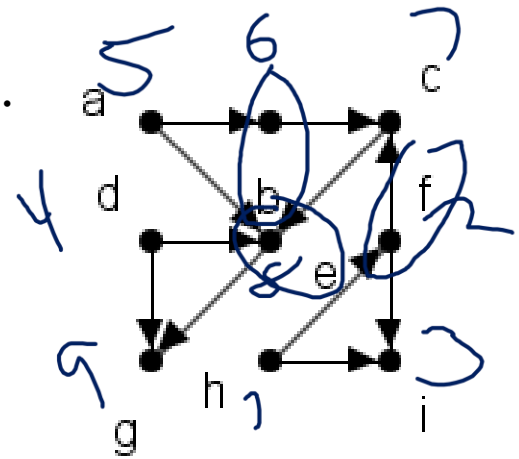


  - In reverse, v appears before everything that depends on it

Topological sorting exercise

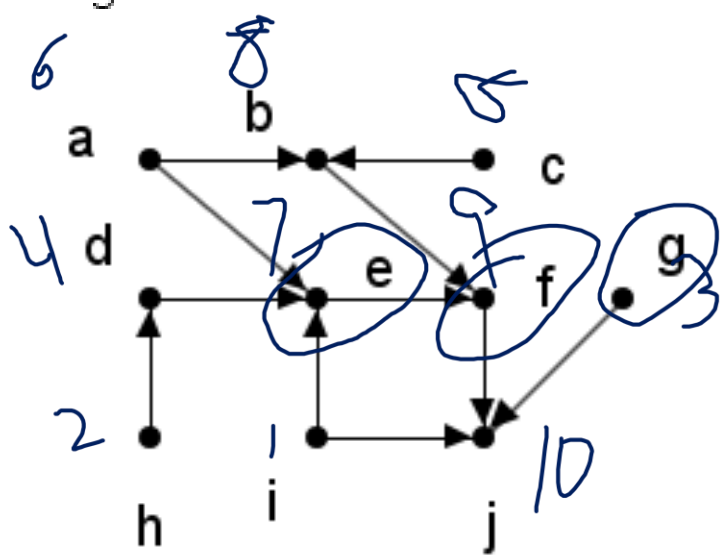
Topologically sort the following DAGs:

1.



Disc: A B C E G D F I A
Exp: G E C B A D I F H
Topological: H F I D A B C E G

2.

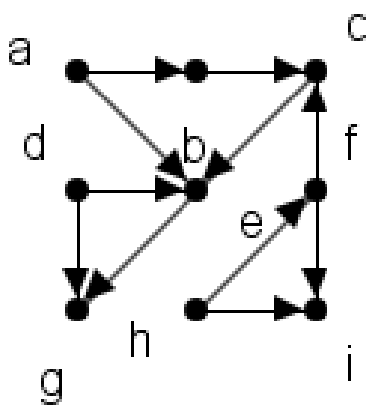


Disc: A B F J E C D G H I
Exp: J F B E A C D G H I
Topo: I H G D C A E B F J

Topological sorting exercise

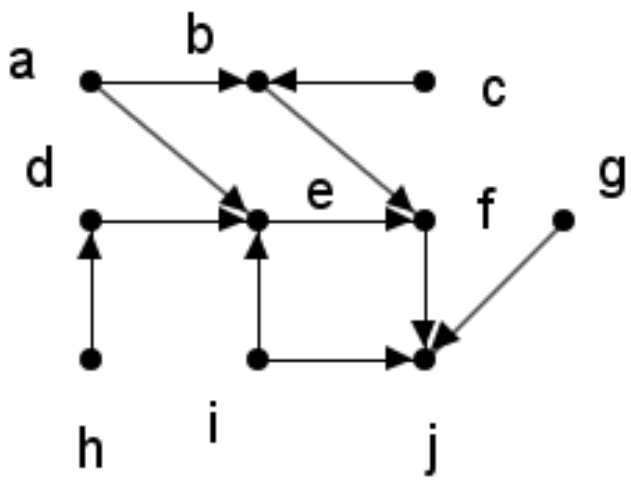
Topologically sort the following DAGs:

1.



DFS: g, e, c, b, a, d, i, f, h
Sorted: h, f, i, d, a, b, c, e, g

2.



DFS: j, f, b, e, a, c, d, g, h, i
Sorted: i, h, g, d, c, a, e, b, f ,j

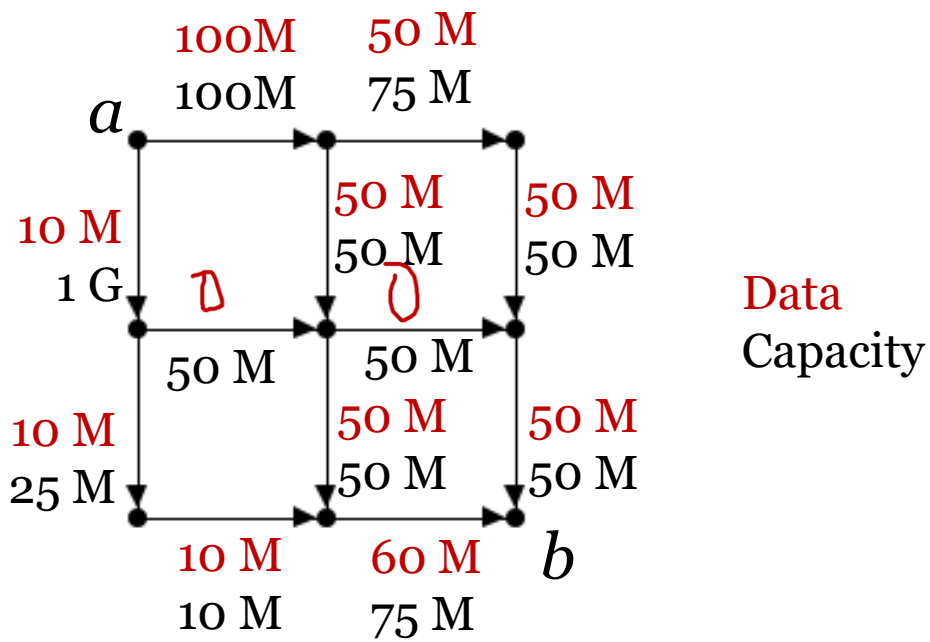
Source-sink networks

- Networks that model information or material flow
- Weighted, directed graph with two special nodes
  - Source: vertex with *in-degree* 0
  - Sink: vertex with *out-degree* 0
  - All weights are positive
    - Represent edge *capacity*
    - Max data or material that can be transported

- **Network flow:** assignment of values to edges such that:
  - Flow never exceeds capacity
  - In-flow equals out-flow everywhere except source and sink
- **Value of a flow:** total in-flow at sink
  - How much data/material the entire network transports
  - Denoted val(f)

Source-sink network example

- The network below can transmit 110 megabits (Mbps) per second between *a* and *b*:



- No link exceeds capacity
- 110 Mbps leave *a* (100 + 10)
- 110 Mbps arrive at *b* (50 + 60)
- Not possible to push more data through network (*later*)

Augmenting paths

Definition. An *augmenting path* is an undirected source-sink path for a flow f such that:

1. If (u, v) is oriented towards the sink, $f(u, v) < cap(u, v)$.
2. If (u, v) is oriented towards the source, $f(u, v) > 0$.

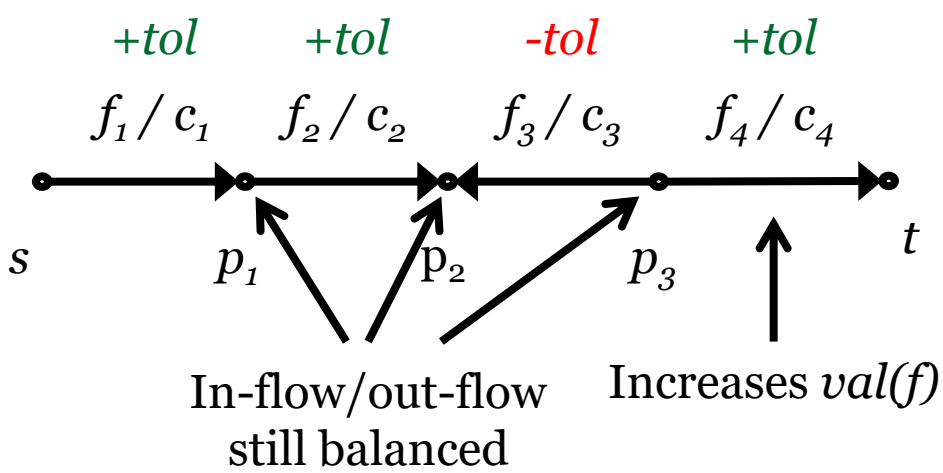
The *tolerance* of a flow is the minimum value of $cap(u, v) - f(u, v)$ for the "forwards" edges and $f(u, v)$ for the "backwards" edges.

Key Ideas

"Forwards" edges are underutilized

"Backwards" edges pull flow from sink

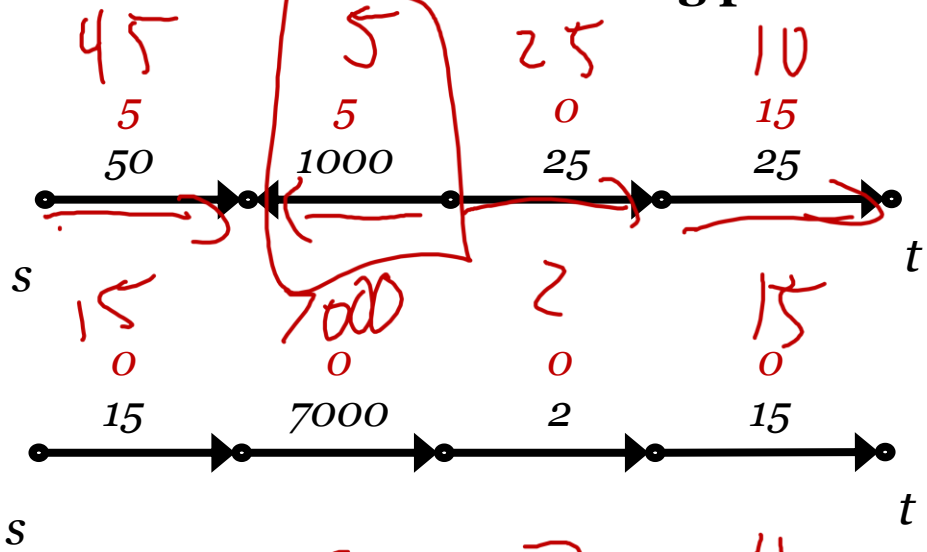
Tolerance is how much flow you can "add"



Augmenting path examples

What are the tolerances of the following paths?

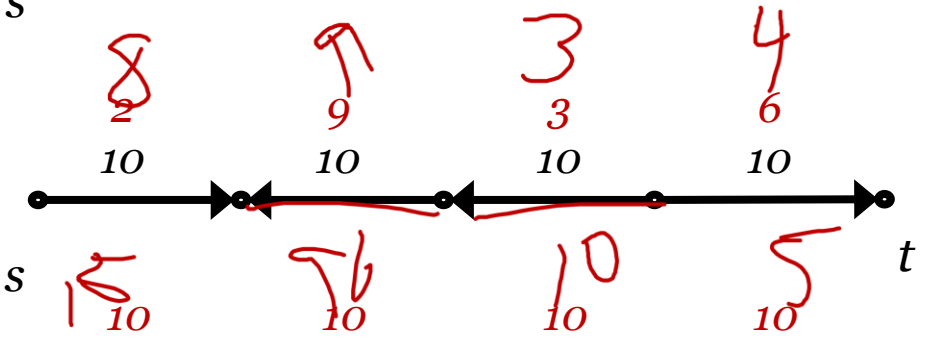
1)



5

5

2)



2

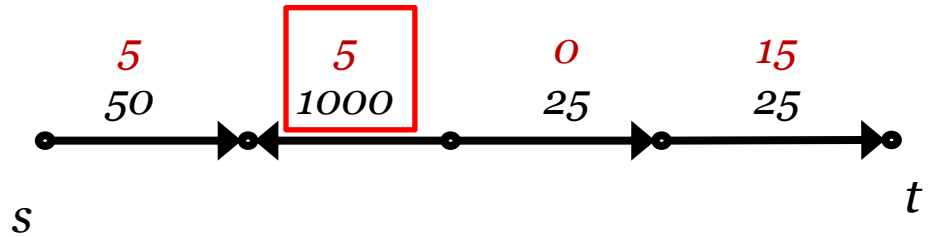
3)

3

5

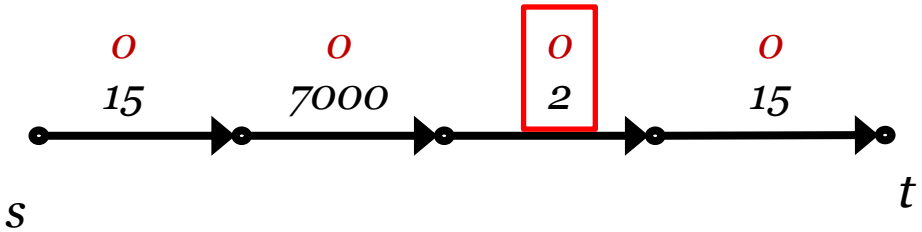
Augmenting path examples

What are the tolerances of the following paths?



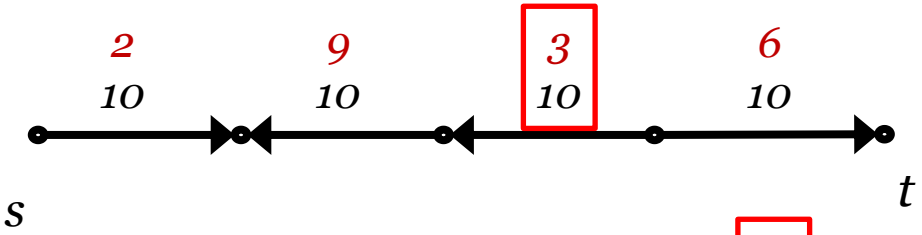
5

1)



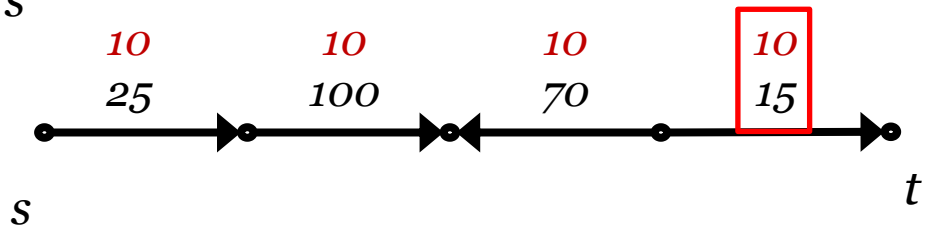
2

2)



3

3)



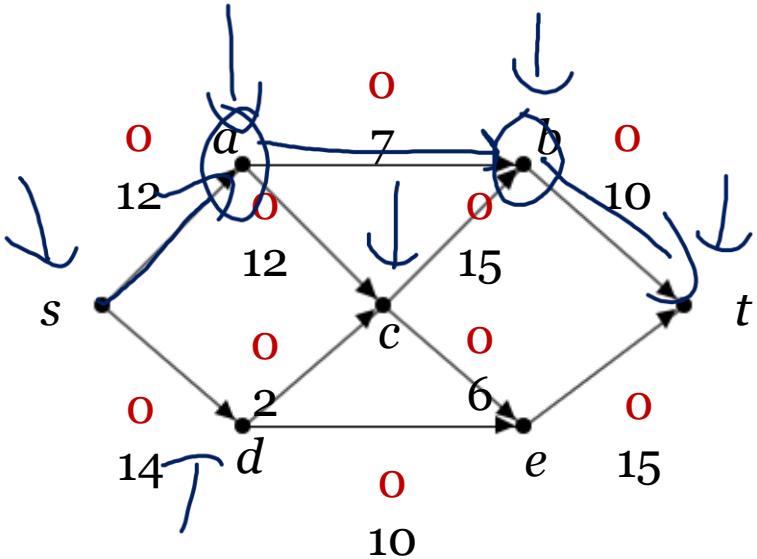
5

Ford-Fulkerson algorithm

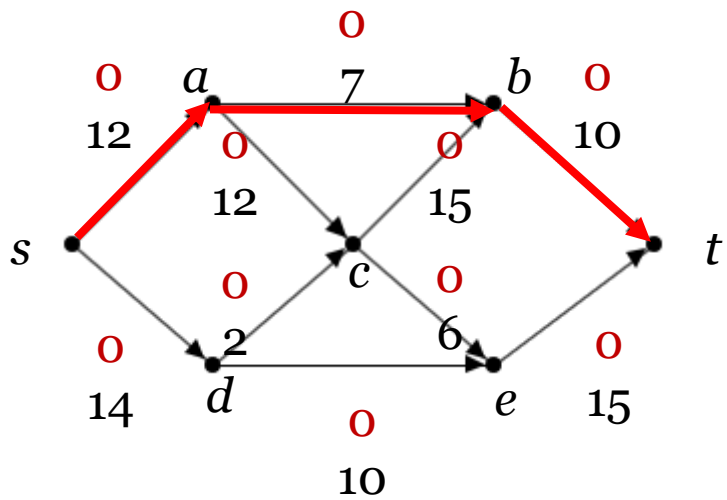
Pseudocode

1. Set all edges to have flow 0
   - Guaranteed to be valid
2. Start at source
3. Mark all vertices with positive tolerance
4. Choose an unvisited marked vertex
5. Repeat steps 3 & 4 until you reach sink or can't continue
   - If you reach sink:
     - path from source is an augmenting path
   - If you run out of vertices:
     - none of the outgoing edges can be increased
     - all incoming edges have zero flow
6. Increase flow and restart from 2 if you reached the sink
7. Otherwise, return set of reachable vertices
   - All vertices with positive tolerance

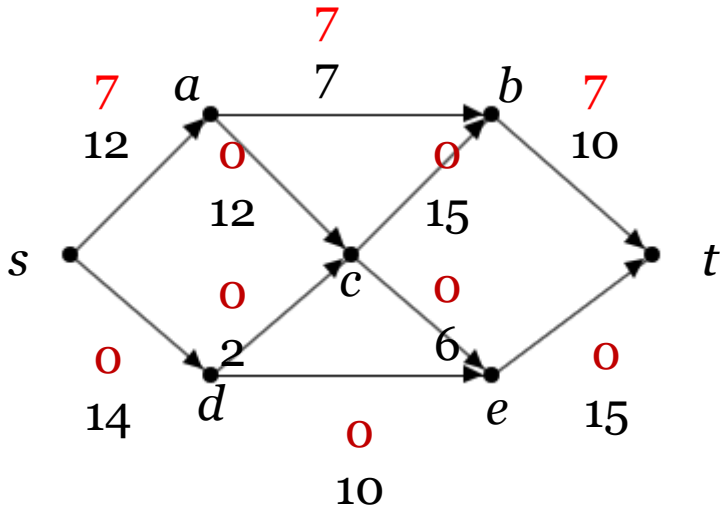
Ford-Fulkerson example



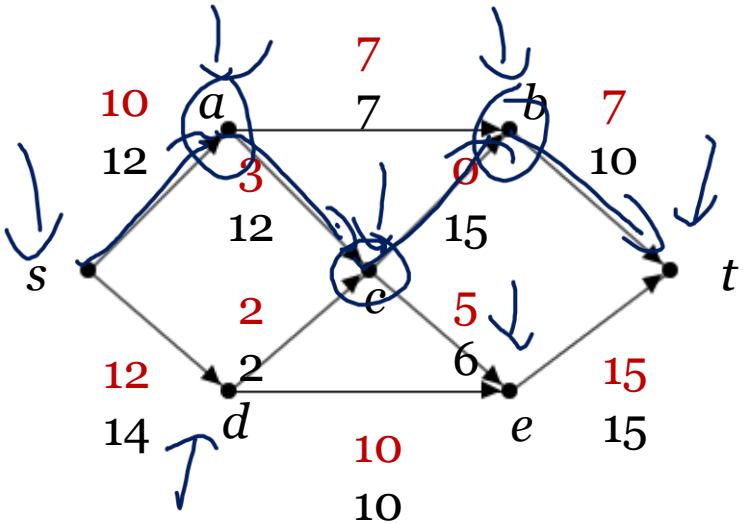
Ford-Fulkerson example



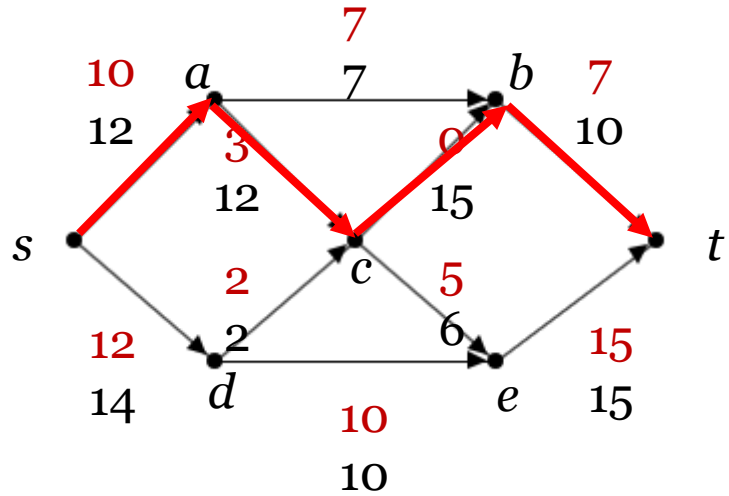
Tolerance = 7



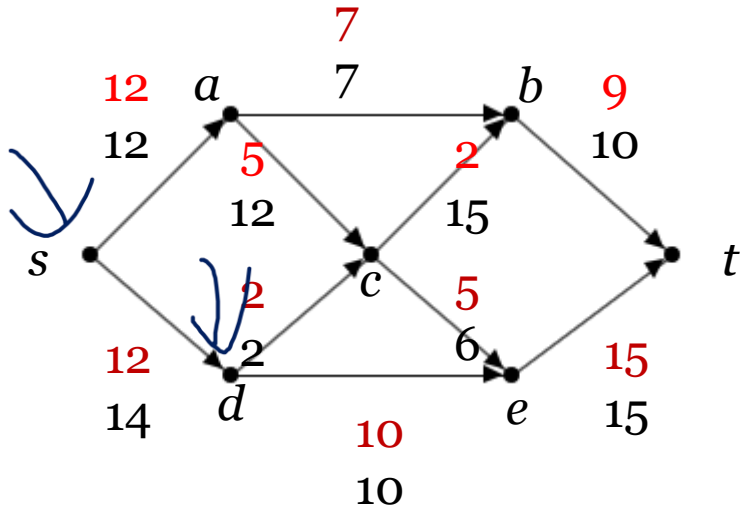
Later Ford-Fulkerson example



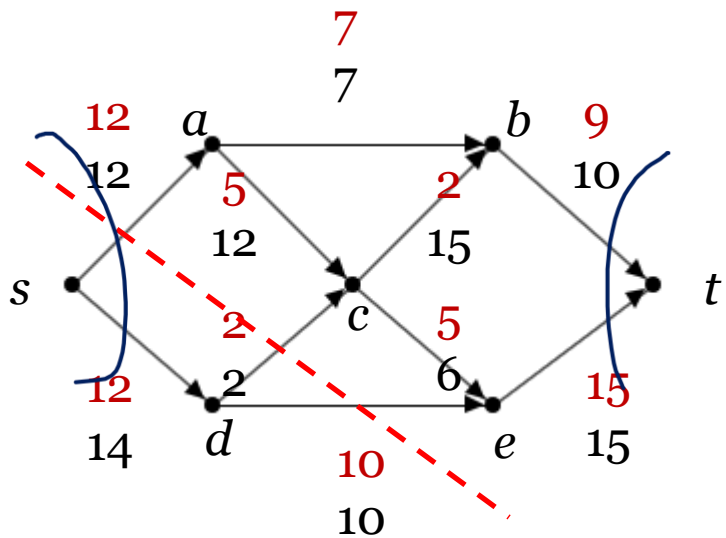
Later Ford-Fulkerson example



Tolerance = 2



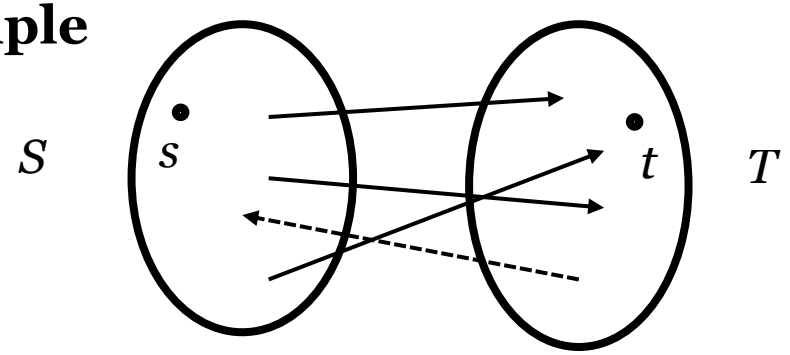
Later Ford-Fulkerson example



All outgoing edges have flow = capacity,
all incoming edges have flow = 0

Source-sink cuts

- Network partition (S, T) such that:
  1. every vertex belongs to S or T
  2. S and T do not overlap
  3. the source belongs to S
  4. the sink belongs to T
- **Min-cut:** source sink cut with the minimum *capacity*
  - **Capacity of a cut:** sum of capacities of edges <u>from S to T</u>
  - Represents the "cost" of disconnecting sink and source
  - Strongly related to network flows
- **Example**



  - Dotted edge does not contribute to cap(S, T)

Max-flow, min-cut theorem

Theorem. *The value of a maximum flow equals the capacity of a minimum cut.*

Lemma. *For any flow f and cut (S,T), the net flow from S to T equals the value of the flow; i.e.,*

val 0

$$f(S,T) - f(T,S) = val(f)$$

Proof not shown

Corollary. *Any flow's value must be less than any cut's capacity*

Proof. $val(f) = f(S,T) - f(T,S) \leq cap(S,T) - 0$

Proof. When Ford-Fulkerson ends*, $f(e) = cap(e)$ for $S-T$ edges, and $f(e) = 0$ for $T-S$ edges, so

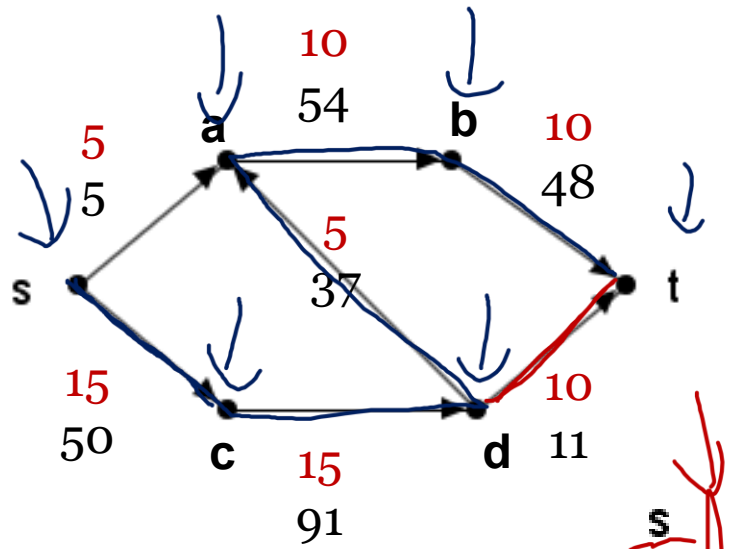
$$\begin{aligned} val(f) &= f(S,T) - f(T,S) \\ &= cap(S,T) - 0 \\ &= cap(S,T) \end{aligned}$$

This must be the max-flow *and* min-cut. □

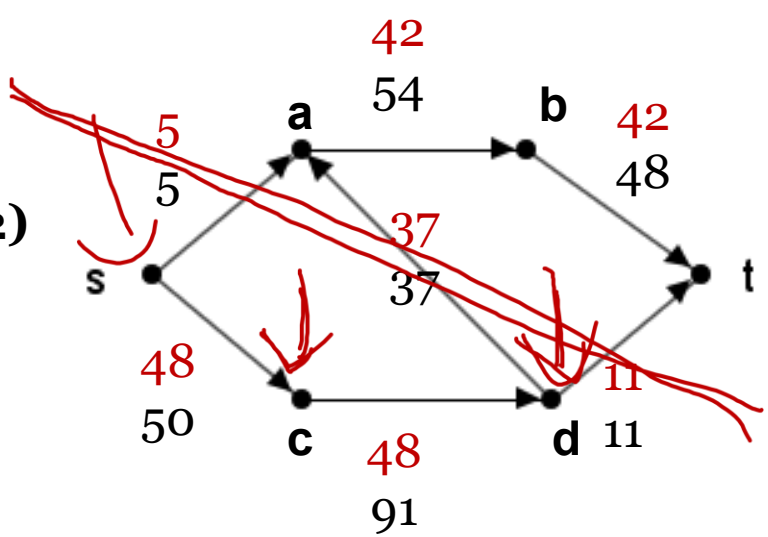
Ford-Fulkerson exercises

Find an augmenting path or min cut for the following network flows:

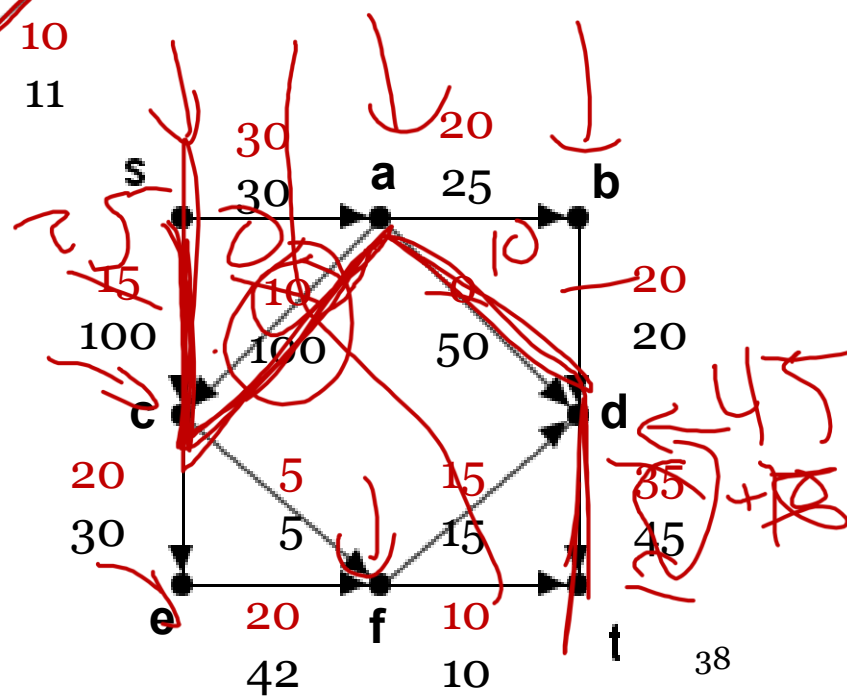
1)



2)



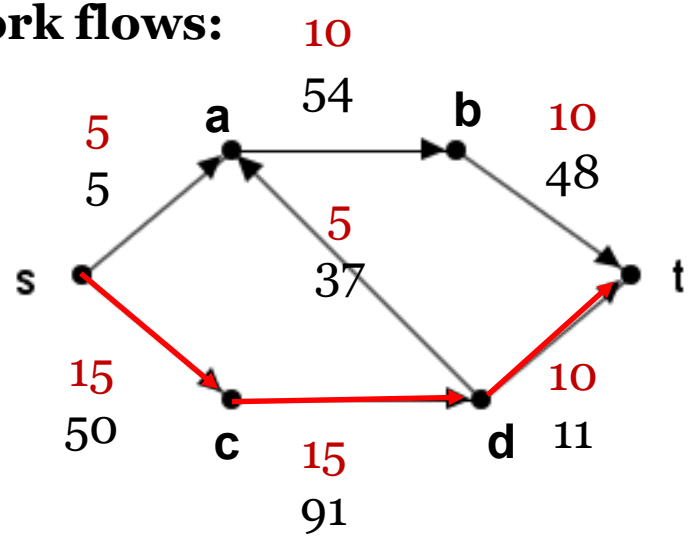
3)



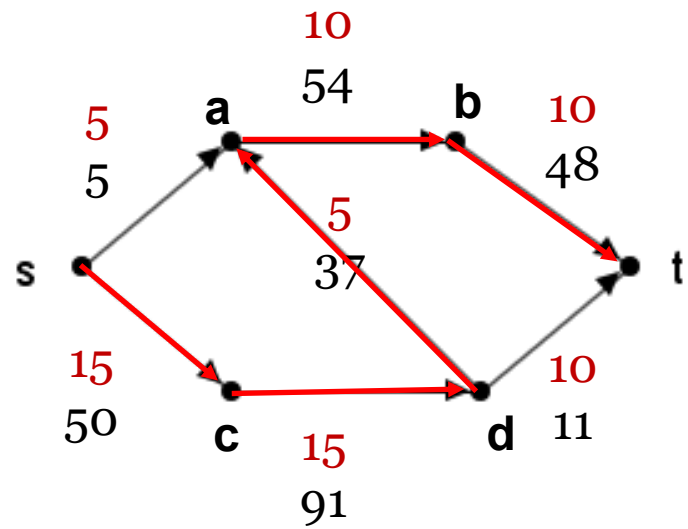
Ford-Fulkerson exercises

Find an augmenting path or min cut for the following network flows:

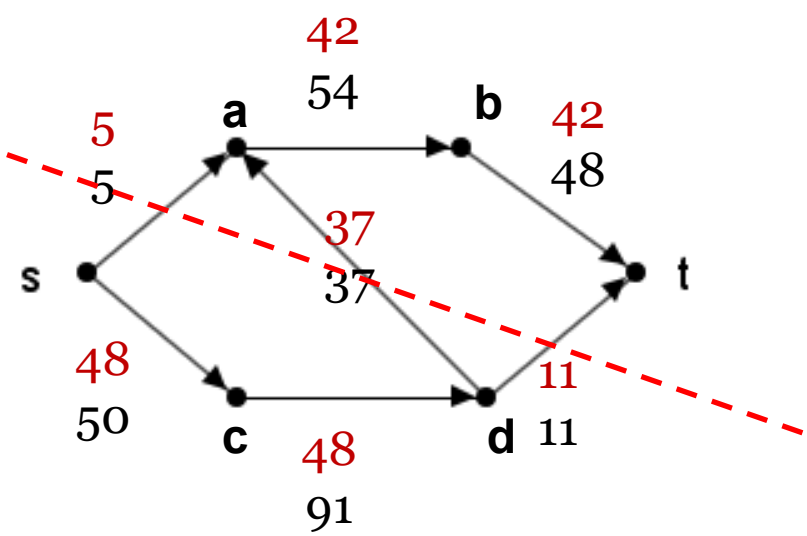
1)



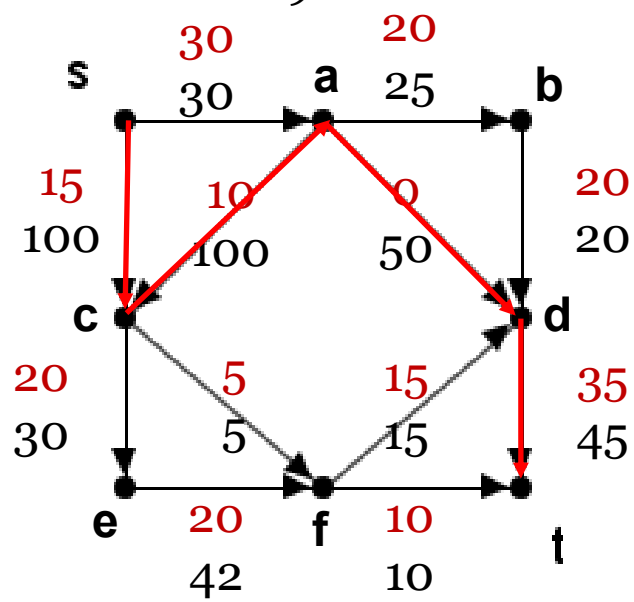
or



2)

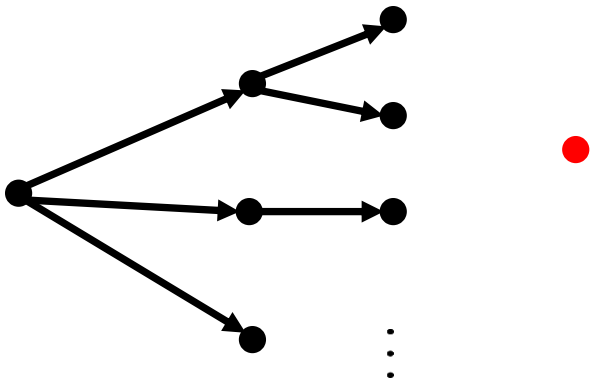


3)



Backtracking

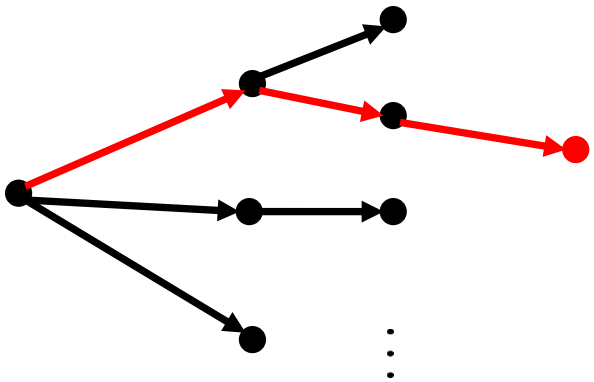
- Algorithm strategy based on graph traversal
  - Can be exact or heuristic



- **Main idea**
  - Model problem as sequence of decisions
  - Set of states and decisions defines a graph
  - Traverse with DFS

Backtracking

- Algorithm strategy based on graph traversal
  - Can be exact or heuristic



- **Main idea**
  - Model problem as sequence of decisions
  - Set of states and decisions defines a graph
  - Traverse with DFS

- **Complexity:** $\Theta(n+m) = \Theta(n)$ if tree

- How big is this graph?
  - Contains all possible solutions
  - Usually exponential

Improving naïve backtracking

- Don't construct entire state graph!
  - Keep track of location and candidates
- Don't revisit same state
- Don't explore states that can't lead to the solution
  - *Pruning* or *branch and bound*

- How you explore search space matters
  - Best order depends on problem
- Explore promising candidates first
  - Stop when you know you've found solution
  - E.g., A*
- Explore candidates that eliminate the most possibilities first
- Heuristic backtracking
  - Only explore most likely candidates
  - E.g., beam search

Coming up

- Backtracking
- Intro to Complexity Theory

- **Recommended readings:** Sections 16.1-16.2.2, 18.4-18.4.2
- *Practice problems*: R-16.2, R-16.4, R-16.7, C-16.3, C-16.5a, R-18.7